Actividad — RORO Supreme

Integración de dos memorias ROM, ALUMIPS y una memoria externa de instrucciones

Objetivo

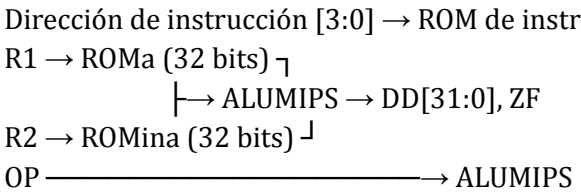
Diseñar el módulo superior RORO Supreme para ejecutar una secuencia de 16 instrucciones de 12 bits. Reutilizar los módulos ROM, ALUMIPS y RORO proporcionados, crear la ROM de instrucciones y desarrollar los testbenches de RORO y del sistema completo.

Funcionamiento

La ROM de instrucciones entrega una palabra de 12 bits. OP selecciona la operación de la ALU; R1 y R2 seleccionan posiciones de ROMa y ROMina. Ambas memorias entregan operandos de 32 bits a la ALUMIPS. Las salidas son DD[31:0] y ZF. La selección de la siguiente instrucción puede realizarse mediante una dirección de 4 bits controlada por el testbench; no es necesario implementar un contador de programa.

Formato: OP[11:8] | R1[7:4] | R2[3:0]. R1 y R2 tienen 4 bits; si reutilizan la ROM entregada de 6 bits de dirección, deben extender la dirección con dos ceros.

Diagrama lógico



Operaciones obligatorias

De acuerdo con el testbench ALUMIPS compartido: OP=0010 suma, OP=0110 resta, OP=0111 SLT (comparación menor que). No se exige otra operación sin verificar primero la codificación real del módulo ALUMIPS proporcionado.

Programa obligatorio: 16 instrucciones, distribuidas en 6 sumas, 6 restas y 4 comparaciones SLT. Usar las 16 direcciones de ROMa y ROMina. Los archivos adjuntos proporcionan datos de referencia y una propuesta de instrucciones; cada estudiante debe explicar la codificación y comprobar los resultados.

Desarrollo

1. 1. Inspecciona las interfaces de ROM, ALUMIPS y RORO proporcionadas. No modifiques su lógica sin justificarlo.
2. 2. Construye dos instancias ROM para ROMa y ROMina, cargadas desde archivos separados mediante $readmemb.
3. 3. Diseña una ROM de instrucciones de 16 posiciones y palabras de 12 bits, inicializada desde archivo mediante $readmemb.
4. 4. Diseña el módulo superior RORO Supreme, conectando la instrucción, las dos ROM y la ALU.
5. 5. Escribe RORO_tb para verificar individualmente el bloque de dos ROM + ALU.
6. 6. Escribe RORO_Supreme_tb para recorrer las 16 instrucciones y comprobar DD y ZF.
7. 7. Documenta las instrucciones en una tabla: índice, OP, R1, R2, operandos, DD esperado y ZF esperado.

Plan de pruebas obligatorio

El testbench debe recorrer las 16 direcciones de instrucción, comparar el resultado observado contra el esperado y reportar PASS/FAIL por caso. Incluye al menos: suma con resultado distinto de cero; suma con cero; resta positiva; resta negativa (interpretación de 32 bits); resta con resultado cero y ZF=1; SLT verdadero y falso. Los valores de referencia del CSV contienen los 16 casos exactos.

Dir.	OP	ROMa	ROMina	DD esperado
0	2	12	5	17
1	2	7	14	21
2	2	25	10	35
3	2	3	9	12
4	2	40	20	60
5	2	18	30	48
6	6	0	0	0
7	6	15	6	9
8	6	63	12	51
9	6	9	50	4294967255
10	6	100	3	97
11	6	2	2	0
12	7	31	31	0
13	7	55	11	0
14	7	8	16	1
15	7	20	4	0

ZF debe ser 1 únicamente cuando DD sea cero. Las restas negativas se comparan como patrones de 32 bits en complemento a dos.

Entregables

- Módulo superior RORO Supreme y ROM de instrucciones, escritos por el estudiante.
- RORO_tb.v y RORO_Supreme_tb.v con comprobaciones automáticas PASS/FAIL.
- Archivos de inicialización ROMa, ROMina e instrucciones (12 bits × 16 posiciones).
- Tabla de 16 instrucciones y resultados esperados.
- Captura o registro de simulación y explicación breve de dos casos (una resta negativa y ZF=1).

Consideraciones técnicas

La ROM.v proporcionada declara 64 posiciones de 32 bits y usa un nombre fijo de archivo initData.txt. Para tener ROMa y ROMina con datos diferentes deberán adaptar o parametrizar la carga de archivos, conservando la funcionalidad del módulo. Los archivos ROMa_init.txt y ROMina_init.txt tienen 64 líneas para ser compatibles con esa profundidad; sólo se utilizan las primeras 16 direcciones. El código RORO.v previamente compartido integra ROM + RAM; el nuevo RORO de esta actividad será ROM + ROM + ALUMIPS, según el esquema solicitado.

Entregables

Esta actividad se realizará **en equipo**. Cada equipo deberá crear una **organización en GitHub**, que representará a su equipo de trabajo, y dentro de ella un repositorio exclusivo para el proyecto RORO Supreme.

1. RORO_tb.v — Testbench para verificar el funcionamiento de RORO.
2. RORO_Supreme.v — Módulo principal que integra RORO con la memoria ROM de instrucciones. (Como este necesita ROM.v, ALUMIPS.v y RORO.v, incluirlos)
3. RORO_Supreme_tb.v — Testbench que verifica la ejecución de las 16 instrucciones.
4. ROM_Instrucciones.v — Memoria ROM de 16 posiciones y 12 bits por instrucción.
5. instrucciones.txt — Archivo de inicialización de la ROM mediante $readmemb, con las 16 instrucciones correspondientes.
6. **Evidencia de simulación:** demostrar el funcionamiento de las 16 operaciones solicitadas (6 sumas, 6 restas y 4 comparaciones SLT).
7. **Repositorio de GitHub:** enlace al repositorio del proyecto, alojado dentro de la organización del equipo. (el README debe ser el reporte de esta actividad, basado en la guía de reportes)

Evaluación (100 puntos)

Integración y conexiones: 20; ROM de instrucciones y formato: 20; RORO_tb: 15; RORO_Supreme_tb y 16 comprobaciones: 25; Reporte con evidencia y explicación: 20.

Consideraciones:

- Los módulos ROM, ALUMIPS y RORO son los que desarrollamos durante las clases. Deberán reutilizarlos e integrarlos en este proyecto.
- Todos los integrantes deberán pertenecer a la organización y tener acceso al repositorio.
- Cada integrante deberá realizar aportaciones identificables mediante sus propios commits.
- El repositorio deberá contener todos los archivos necesarios para compilar y simular el proyecto.
- Todos los integrantes deberán ser capaces de explicar el funcionamiento general del sistema y las contribuciones que realizaron.

Entrega: Un solo repositorio por equipo. Compartir el enlace de GitHub e indicar los nombres de todos sus integrantes.